UNIVERSITÉ MUNDIAPOLIS
École d'Ingénieurs
Casablanca

2ème Année CI
Génie Informatique
Filière : Informatique

Principes de la Blockchain

Application des principes de la blockchain en Python

Module : Blockchain — TP 01
Hachage SHA-256, chaînage de blocs et minage

Réalisé par :
FELLAH Youssef

Encadré par :
Prof. LOUJA Ayoub

Année universitaire : 2026/2027

Table des matières

1 Introduction **3**
1.1 Contexte . . . 3
1.2 Objectifs . . . 3
1.3 Outils . . . 3
1.4 Registre de transactions . . . 3

2 Application **3**
2.1 Fonction de hachage SHA-256 . . . 4
2.2 Représentation d'un bloc et chaînage . . . 4
2.3 Minage par preuve de travail . . . 5
2.4 Visualisation de la chaîne . . . 7
2.5 Vérification de l'intégrité . . . 7
2.6 Analyse : pourquoi falsifier est coûteux . . . 8

3 Synthèse **8**
3.1 Bilan . . . 8
3.2 Limites de l'implémentation . . . 8
3.3 Améliorations possibles . . . 9

4 Annexes & Bibliographie **9**
4.1 Annexe A : code complet . . . 9
4.2 Annexe B : glossaire . . . 9
4.3 Bibliographie . . . 9

Table des figures

1 Chaînage : le champ `prev` d'un bloc recopie le `hash` du bloc précédent (valeurs illustratives) 5
2 Une modification dans un bloc ancien invalide tous les blocs suivants . . 8

1 Introduction

1.1 Contexte

Une **blockchain** est un registre de transactions partagé, dans lequel les enregistrements sont regroupés en *blocs* reliés les uns aux autres par cryptographie. Deux propriétés la rendent intéressante :

- **l'intégrité** : toute modification d'une donnée déjà enregistrée est détectable ;
- **le coût de la falsification** : réécrire l'historique demande un effort de calcul volontairement élevé (preuve de travail).

Ce rapport présente une implémentation pédagogique de ces mécanismes en Python, réalisée dans le cadre du TP 01 du module Blockchain.

1.2 Objectifs

Le travail réalisé vise à :

1. calculer une empreinte **SHA-256** avec la bibliothèque `hashlib` ;
2. représenter un bloc par un **dictionnaire** et relier les blocs par leur empreinte pour former une chaîne ;
3. programmer le **minage** par preuve de travail (*Proof of Work*) ;
4. **vérifier l'intégrité** de la chaîne.

1.3 Outils

Le code est écrit en Python 3 dans un notebook Jupyter (`main.ipynb`). Seule la bibliothèque standard `hashlib` est utilisée.

1.4 Registre de transactions

Les mêmes cinq enregistrements sont utilisés tout au long du travail. Ils sont écrits sans accents, pour que le texte haché soit identique sur toutes les machines.

Bloc	Donnée enregistrée
0	`Bloc de genese Mundiapolis`
1	`Ali envoie 12$ a Sara`
2	`Sara envoie 5$ a Karim`
3	`Karim envoie 3$ a Nadia`
4	`Nadia envoie 7$ a Ali`

TABLE 1 – *Registre de transactions du TP*

2 Application

2.1 Fonction de hachage SHA-256

Principe

Une fonction de hachage cryptographique transforme un texte de taille quelconque en une **empreinte** de taille fixe. SHA-256 produit 256 bits, soit $256/4 = 64$ caractères hexadécimaux. Elle possède trois propriétés utilisées dans la suite :

- **déterminisme** : le même texte donne toujours la même empreinte ;
- **taille fixe** : la sortie fait 64 caractères, quelle que soit l'entrée ;
- **effet avalanche** : une modification minime du texte change presque toute l'empreinte, ce qui empêche de prévoir le hash d'un texte modifié.

Algorithme

La fonction `calculer_sha256` suit trois étapes : encoder le texte en octets, appliquer SHA-256, puis renvoyer le résultat en hexadécimal.

```python
import hashlib

def calculer_sha256(texte):
    texte_bytes = texte.encode()               # texte -> octets
    hash_object = hashlib.sha256(texte_bytes) # calcul SHA-256
    return hash_object.hexdigest()             # 64 caracteres hexa
```

Résultats

Le hash de `"Blockchain"`, calculé deux fois, est identique (déterminisme) et mesure 64 caractères :

```
625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1
```

Mise en évidence de l'effet avalanche

La fonction `compare_hashes` parcourt les deux empreintes position par position et compte les caractères différents. On l'applique à deux transactions qui ne diffèrent que d'un seul caractère :

`Ali envoie 12$ a Sara` vs `Ali envoie 13$ a Sara`

Les deux empreintes diffèrent sur **62 caractères sur 64**. Sans lien entre les deux, deux chaînes hexadécimales aléatoires diffèrent en moyenne sur $64 \times \frac{15}{16} = 60$ positions : le résultat observé est cohérent avec ce comportement. Changer un caractère d'une transaction produit donc un hash complètement différent.

2.2 Représentation d'un bloc et chaînage

Structure d'un bloc

Un bloc est un dictionnaire Python à cinq clés :

Clé	Contenu
`index`	position du bloc dans la chaîne (0, 1, 2, …)
`data`	la transaction enregistrée
`nonce`	compteur modifié par le minage
`previous_hash`	hash du bloc précédent
`hash`	hash du bloc lui-même

TABLE 2 – *Les cinq champs d'un bloc*

Principe du chaînage

Le hash d'un bloc est calculé à partir de la concaténation

$$\text{hash} = \text{SHA-256}\big(\text{previous_hash} \,\|\, \text{data} \,\|\, \text{nonce}\big).$$

Comme `previous_hash` fait partie du texte haché, le hash d'un bloc *dépend* de celui de son prédécesseur, qui dépend lui-même du précédent, et ainsi de suite jusqu'au bloc de genèse. C'est ce mécanisme qui forme la chaîne (figure 1). Le premier bloc, dit **bloc de genèse**, n'a pas de prédécesseur : on lui attribue un `previous_hash` fixe composé de zéros.

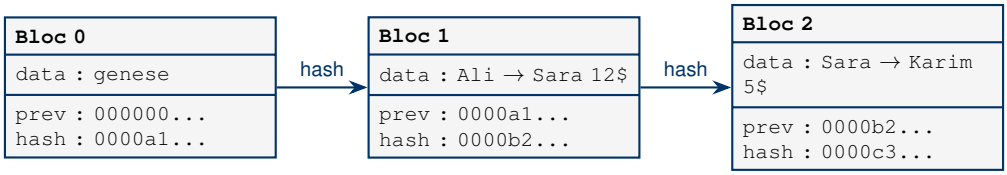


FIGURE 1 – *Chaînage : le champ `prev` d'un bloc recopie le `hash` du bloc précédent (valeurs illustratives)*

Algorithmes

- `creer_bloc(chain, data, difficulty)` : l'index est la taille actuelle de la chaîne ; `previous_hash` est celui du dernier bloc (ou la valeur fixe de genèse si la chaîne est vide) ; le nonce et le hash sont obtenus par le minage ;
- `add_bloc(chain, data, difficulty)` : crée le bloc puis l'ajoute à la fin de la liste avec `append`.

2.3 Minage par preuve de travail

Principe

Calculer un hash est presque instantané : sans contrainte supplémentaire, n'importe qui pourrait réécrire toute une chaîne sans effort. La **preuve de travail** ajoute une contrainte : le hash d'un bloc n'est accepté que s'il commence par un nombre fixé de zéros, appelé **difficulté**.

Comme un hash se comporte comme un tirage aléatoire, il n'existe pas de méthode plus rapide que l'essai répété. C'est le rôle du **nonce** : un entier que l'on fait varier

$(0, 1, 2, \dots)$ jusqu'à obtenir un hash valide. Chaque essai réussit avec la probabilité 16^{-k} pour une difficulté de k zéros (un caractère hexadécimal a 16 valeurs possibles). Le nombre moyen d'essais est donc

$$N_{\text{moyen}} = 16^k, \qquad \text{soit } 16^6 = 16\,777\,216 \text{ essais pour } k = 6.$$

Chaque zéro supplémentaire multiplie ce coût par 16.

Algorithme de `calculer_hash_bloc`

1. initialiser le nonce (à -1 dans la fonction appelante) ;
2. **répéter** : augmenter le nonce de 1 ; calculer `SHA-256(previous_hash + data + nonce)` ;
3. compter combien des `difficulty` premiers caractères valent `"0"` ;
4. **s'arrêter** lorsque ce compte est égal à `difficulty`, et renvoyer le couple (hash, nonce).

```python
def calculer_hash_bloc(prev_hash, data, nonce, difficulty):
    valid = False
    while valid == False:
        nonce += 1
        hash = calculer_sha256(str(prev_hash) + str(data) + str(nonce))
        zeros = 0
        for i in range(difficulty):
            if hash[i] == "0":
                zeros += 1
        if zeros == difficulty:
            valid = True
    return (hash, nonce)
```

Construction de la chaîne

On parcourt la liste des cinq transactions et on appelle `add_bloc` pour chacune, avec une difficulté de 6. Le tableau 3 donne les résultats obtenus.

Bloc	Donnée	Nonce	Début du hash
0	Bloc de genese Mundiapolis	4 094 026	`000000531c2d682c...`
1	Ali envoie 12$ a Sara	760 243	`0000007b31104a0b...`
2	Sara envoie 5$ a Karim	6 547 682	`000000d0085f24d8...`
3	Karim envoie 3$ a Nadia	6 918 157	`0000008168f83d14...`
4	Nadia envoie 7$ a Ali	55 740 373	`000000d156a8781b...`

TABLE 3 – *Blocs minés avec une difficulté de 6 zéros*

Les nonces varient fortement d'un bloc à l'autre (de $7{,}6 \times 10^5$ à $5{,}6 \times 10^7$) : le minage est un processus aléatoire dont seule la moyenne, $16^6 \approx 1{,}7 \times 10^7$, est prévisible.

2.4 Visualisation de la chaîne

La fonction `afficher_blocs` parcourt la liste et affiche, pour chaque bloc, son index, sa donnée, son nonce, son `previous_hash` et son hash, dans un cadre de caractères. Une flèche verticale est tracée entre deux blocs consécutifs, sauf après le dernier. Cet affichage permet de contrôler visuellement que le champ `previous_hash` de chaque bloc est égal au hash du bloc qui le précède.

2.5 Vérification de l'intégrité

Principe

Vérifier une chaîne consiste à **refaire les calculs** et à comparer avec ce qui est stocké. Pour chaque bloc i, trois contrôles sont nécessaires :

1. **cohérence du hash** : recalculer SHA-256(previous_hash ‖ data ‖ nonce) avec le nonce *stocké* et vérifier qu'il est égal au hash stocké ;
2. **preuve de travail** : le hash commence bien par `difficulty` zéros ;
3. **chaînage** : pour $i > 0$, le `previous_hash` du bloc i est égal au hash du bloc $i - 1$; pour $i = 0$, c'est la valeur fixe de genèse.

Dès qu'un contrôle échoue, la chaîne est déclarée invalide. Notons que la vérification est **bien plus rapide que le minage** : un seul calcul de hash par bloc suffit, car le nonce est déjà connu. C'est cette asymétrie (difficile à produire, facile à vérifier) qui fait l'intérêt de la preuve de travail.

Version finale de l'algorithme

```python
def verify_blockchain(chain, difficulty):
    cible = "0" * difficulty
    for i in range(len(chain)):
        bloc = chain[i]
        # 1) recalcul du hash avec le nonce stocke (pas de minage)
        h = calculer_sha256(str(bloc["previous_hash"]) + str(bloc["data"
            ])
                            + str(bloc["nonce"]))
        if h != bloc["hash"]:
            print(f"Le hash du bloc {i} est invalide.")
            return False
        # 2) preuve de travail
        if not h.startswith(cible):
            print(f"Bloc {i} : la preuve de travail n'est pas satisfaite.
                ")
            return False
        # 3) chainage
        if i > 0 and bloc["previous_hash"] != chain[i - 1]["hash"]:
            print(f"Le hash precedent du bloc {i} est invalide.")
            return False
    return True
```

2.6 Analyse : pourquoi falsifier est coûteux

Supposons qu'un utilisateur modifie la donnée du bloc 1 (par exemple, `12$` devient `2500$`) sans toucher au reste de la chaîne.

1. Par l'effet avalanche, le hash recalculé du bloc 1 est totalement différent du hash stocké : le premier contrôle échoue, la falsification est détectée.
2. Pour la masquer, il faut **re-miner** le bloc 1 afin de retrouver un hash valide avec un nouveau nonce.
3. Mais ce nouveau hash est différent de l'ancien : le `previous_hash` du bloc 2 ne correspond plus (contrôle 3). Il faut donc recopier le nouveau hash dans le bloc 2 et le re-miner à son tour, puis le bloc 3, et ainsi de suite.

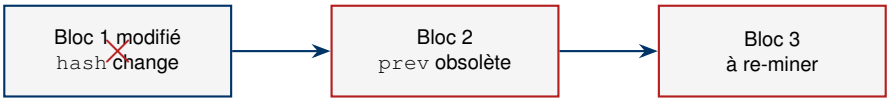


FIGURE 2 – *Une modification dans un bloc ancien invalide tous les blocs suivants*

Conclusion : falsifier le bloc i coûte le re-minage des blocs i à $n-1$, soit environ $(n-i) \times 16^k$ essais. Plus la transaction est ancienne, plus la falsification est chère, et chaque zéro de difficulté en plus multiplie ce coût par 16. À l'inverse, modifier uniquement le dernier bloc ne coûte qu'un seul re-minage : c'est pourquoi, en pratique, les transactions récentes sont considérées comme moins fiables que les anciennes.

3 Synthèse

3.1 Bilan

Ce travail a permis de mettre en pratique les trois mécanismes à la base d'une blockchain :

- le **hachage SHA-256**, dont l'effet avalanche rend toute modification détectable ;
- le **chaînage** : en incluant le hash du bloc précédent dans le calcul du hash, chaque bloc engage tout l'historique qui le précède ;
- la **preuve de travail**, qui rend la production d'un bloc coûteuse (16^k essais en moyenne) alors que sa vérification reste immédiate.

La combinaison de ces mécanismes rend l'historique *pratiquement infalsifiable* : réécrire un bloc ancien oblige à re-miner toute la fin de la chaîne.

3.2 Limites de l'implémentation

L'implémentation reste pédagogique :

- il n'y a **qu'une seule copie** de la chaîne et aucun réseau : un attaquant qui re-mine tous les blocs et qui possède l'unique copie réussit sa falsification. La sécurité d'une vraie blockchain repose aussi sur la **décentralisation** (consensus de la chaîne la plus longue entre de nombreux nœuds) ;

- la **difficulté est fixe** et choisie à la main, alors que les réseaux réels l'ajustent automatiquement ;
- les transactions sont du **texte libre** : il n'y a ni signature numérique ni vérification des soldes ;
- le minage est **séquentiel en Python**, donc très lent comparé à du matériel spécialisé (la difficulté 6 demande déjà plusieurs dizaines de millions d'essais).

3.3 Améliorations possibles

- mesurer le temps de minage pour plusieurs difficultés et vérifier expérimentalement la croissance en 16^k ;
- stocker plusieurs transactions par bloc, avec un arbre de Merkle ;
- ajouter des signatures numériques et un contrôle des soldes ;
- simuler plusieurs nœuds et la règle de la chaîne la plus longue.

4 Annexes & Bibliographie

4.1 Annexe A : code complet

`https://github.com/ind4x/BLOCKCHAIN-LAB/tree/main/TP%20-%2001`

4.2 Annexe B : glossaire

Hash / empreinte
résultat de longueur fixe d'une fonction de hachage.

Nonce
entier que l'on fait varier pendant le minage pour obtenir un hash valide.

Difficulté
nombre de zéros imposés au début du hash.

Bloc de genèse
premier bloc de la chaîne, sans prédécesseur.

Preuve de travail
règle exigeant un effort de calcul pour produire un bloc valide.

4.3 Bibliographie

Références

[1] S. Nakamoto, *Bitcoin : A Peer-to-Peer Electronic Cash System*, 2008.

[2] NIST, *FIPS PUB 180-4 : Secure Hash Standard (SHS)*, 2015.

[3] Python Software Foundation, *hashlib — Secure hashes and message digests*, documentation de la bibliothèque standard Python.

[4] Université Mundiapolis, *TP 01 — Hachage SHA-256, chaînage de blocs et minage*, module Blockchain.